國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab 02

姓名：廖彥瑄

學號：110810030

Lab02-Ex1.
計算程式步驟數(step count)。
Code

```
int findMax(const vector<int>& arr) {
  int stepCount = 0;
  int max = arr[0];
  stepCount++;//讀取 arr[0]
  stepCount++;//結果存入 max
  stepCount++;//迴圈初始化
  for (int i = 1; i < arr.size(); i++) {
    stepCount++;//比較
    stepCount++;//遞增
    stepCount++;//讀取 arr[i]
    stepCount++;//比較
    if (arr[i] > max) {
      stepCount++;//讀取 arr[i]
      stepCount++;//比較
      max = arr[i];
      stepCount++;//讀取 arr[i]
      stepCount++;//結果存入 max
    }
  }
  stepCount++; //迴圈結束
  stepCount++; //程式結束
  cout << "Step count: " << stepCount << endl;
  return max;
}
```

Lab02-Ex2.
計算程式步驟數(step count)。
Code

```
void printPairs(const vector<int>& arr) {
  int stepCount = 0;
  stepCount++;//迴圈初始化
  for (int i = 0; i < arr.size(); i++) {
    stepCount++;//比較
    stepCount++;//遞增
    for (int j = i + 1; j < arr.size(); j++) {
      stepCount++;//加 1
      stepCount++;//結果存入 j
      stepCount++;//比較
      stepCount++;//遞增
      cout << arr[i] << "," << arr[j] << endl;
    }
    stepCount++; //迴圈結束
  }
  stepCount++; //迴圈結束
  cout << "Step count: " << stepCount << endl;
}
```

Lab02-Ex3.

計算程式步驟數(step count)。

Code

(a)

```
void copyArray(const vector<int>&
source, vector<int>& dest) {
  int stepCount = 0;
  stepCount++;//迴圈初始化
  for (int i = 0; i < source.size();
  i++) {
    stepCount++;//比較
    stepCount++;//遞增
    dest[i] = source[i];
    stepCount++;//讀取 source[i]
    stepCount++;//結果存入 dest[i]
  }
  stepCount++; //迴圈結束
```

(b)

```
int countElement(const vector<int>&
arr, int target) {
  int stepCount = 0;
  int count = 0;
  stepCount++;//count 初始化
  stepCount++;//迴圈初始化
  for (int i = 0; i < arr.size(); i++)
  {
    stepCount++;//比較
    stepCount++;//遞增
    if (arr[i] == target) {
    stepCount++;//讀取 arr[i]
    stepCount++;//結果存入 arr[i]
    count++;
```

```
  cout << "Step count: " << stepCount
  << endl;
}
```

```
    stepCount++;//count+1
    }
  }
  stepCount++; //迴圈結束
  stepCount++; //程式結束
  return count;
}
```

Discussion Section

1.

Because it's typically O(1) in C++ vectors ➔ Constant-time access

The size is stored as a member variable ➔ No calculations.

2.

(1)因為 Big-O 是用來表示成長趨勢，這與常數無關

(2)因為成長趨勢只與 n 有關

(3)判斷適合用來處理少量資訊還是大量資訊

3.

(1) sumArray 比較多

(2)不會，因為都只和 n 有關，是線性的